

Password Security Explained Symmetric Encryption (SE), Asymmetric Encryption (ASE), and Hashing

This document explains the difference between encryption and hashing, especially how they are used in password security and Spring Security.

1. The Main Difference: Reversible vs One-Way

The most important question is: Can we recover the original value?

Encryption is reversible. If you have the correct key, you can get the original data back.

Hashing is one-way. You cannot decrypt a hash to get the original password.

2. Symmetric Encryption (SE)

Symmetric encryption uses one secret key for both encryption and decryption.

Flow:

Original Data -> Encryption + Secret Key -> Encrypted Data

Encrypted Data -> Same Secret Key -> Original Data

Example:

Password: Ahmed123

Secret Key: MySecretKey

Encrypted result: X92ks02ks

Later, using the same key, the application can recover Ahmed123.

Advantages of Symmetric Encryption

- Very fast compared with asymmetric encryption.
- Good for encrypting large amounts of data.
- Used in systems where the application needs the original information again.

Disadvantages of Symmetric Encryption

- The secret key must be protected.
- If an attacker gets the database and the key, encrypted data can be exposed.
- Not recommended for storing user passwords.

Common Example: AES

AES (Advanced Encryption Standard) is a popular symmetric encryption algorithm. It is commonly used for files, storage encryption, and secure communication.

3. Asymmetric Encryption (ASE)

Asymmetric encryption uses two mathematically related keys:

1. Public Key: Anyone can have it. It is used for encryption.
2. Private Key: Kept secret. It is used for decryption.

Flow:

Message -> Public Key -> Encrypted Message -> Private Key -> Original Message

Example of Asymmetric Encryption

A website has a public key inside its certificate.

Your browser uses that public key to establish secure communication.

The server uses its private key to prove its identity and protect communication.

Common Example: RSA

RSA is a famous asymmetric encryption algorithm used in certificates, digital signatures, and secure communication systems.

4. Hashing

Hashing converts input data into a fixed-length output called a hash.

Example:

Ahmed123

|

v

Hash Function

|

v

\$2a\$10\$H8x92k...

There is no decrypt operation.

Why Hashing is Used for Passwords

Applications do not need to know the user's real password after registration.

They only need to verify that the password entered during login matches the stored password hash.

Password Registration Flow

User enters:

Password = Ahmed@123

Application:

```
passwordEncoder.encode(password)
```

Database stores:

\$2a\$10\$xY7Hjs92...

The original password is never stored.

Password Login Flow

User enters:

Ahmed@123

Spring Security performs:

```
passwordEncoder.matches(rawPassword, storedHash)
```

Result:

true -> login allowed

false -> login rejected

5. Why BCrypt is Used

BCrypt is a password hashing algorithm designed for passwords.

It includes:

- Salt: random data added before hashing.
- Work factor: controls how expensive hashing is.

The same password can create different hashes because different salts are used.

BCrypt Example in Spring Security

Configuration:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Saving password:

```
user.setPassword(passwordEncoder.encode(password));
```

Checking password:

```
passwordEncoder.matches(inputPassword, databasePasswordHash);
```

6. Comparison Table

Symmetric Encryption:

- Keys: One key
- Reversible: Yes
- Example: AES
- Password storage: No

Asymmetric Encryption:

- Keys: Public + Private
- Reversible: Yes
- Example: RSA
- Password storage: No

Hashing:

- Keys: None
- Reversible: No
- Examples: BCrypt, Argon2, PBKDF2
- Password storage: Yes

Final Rule for Spring Security

Never store plain passwords.

Never encrypt passwords expecting to decrypt them later.

Use password hashing algorithms such as BCrypt or Argon2 and verify using `PasswordEncoder.matches()`.